UNIVERSITÀ DELLA CALABRIA

DIPARTIMENTO DI INGEGNERIA INFORMATICA, MODELLISTICA, ELETTRONICA E SISTEMISTICA

Corso di Laurea Magistrale in Ingegneria Informatica

Architetture e Programmazione dei Sistemi di Elaborazione

DBSCAN su x86: ottimizzazione con istruzioni SIMD (SSE e AVX) e OpenMP

Prof.
Fabrizio Angiulli

Gabriele Hamil
Giovanni Pascuzzi

ANNO ACCADEMICO 23/24

Indice

Indice **1**

Elenco delle figure **3**

Elenco delle tabelle **4**

1 Sommario e Introduzione **5**

1.1 Descrizione del Progetto 5

1.2 Organizzazione della Relazione in Capitoli 7

2 L'algoritmo DBSCAN **9**

2.1 Clustering Basato sulla Densità 9

2.2 L'Algoritmo Originale 10

2.3 Convenzioni Adottate 12

2.4 Formulazione in Due Fasi 13

3 Architettura del Software e Strutture Dati **16**

3.1 Organizzazione dei Moduli 16

3.2 Formato dei File e Disposizione in Memoria 18

3.3 L'Array Dinamico PointList 19

3.4 Il Ball Tree 20

3.5 Interfaccia a Riga di Comando 22

4 Ottimizzazione mediante Istruzioni SIMD **23**

4.1 Dove Intervenire 23

4.2 Il Kernel a 32 Bit con SSE 24

4.3 Il Kernel a 64 Bit con AVX 25

4.4 Il Riempimento delle Etichette 27

4.5 Confronto con il Codice del Compilatore 27

5 Parallelizzazione con OpenMP **29**

5.1 L'Approccio della Versione d'Esame 29

5.2 Parallelizzazione della Prima Fase 30

5.3 Costruzione Parallela del Ball Tree 31

5.4 Perché l'Espansione Resta Sequenziale 31

5.5 Misura del Tempo 32

6 Validazione e Analisi Sperimentale **33**

6.1 Ambiente e Metodologia 33

6.2 Correttezza 34

6.3 Confronto tra le Varianti 35

6.4 Confronto tra 32 e 64 Bit 37

6.5 Il Caso della Versione d'Esame 38

6.6 Scalabilità con i Thread 39

6.7 Scalabilità con il Numero di Punti 40

6.8 Dimensione delle Foglie 41

6.9 Effetto della Dimensionalità 42

6.10 SIMD Manuale e Ottimizzazioni del Compilatore 43

7 Limiti e Sviluppi Possibili **45**

7.1 Memoria Occupata dagli Intorni 45

7.2 La Matrice delle Distanze 46

7.3 Kernel SIMD 46

7.4 Struttura dell'Indice 47

7.5 Precisione della Variante a 32 Bit 47

8 Conclusioni **48**

A Dalla Versione d'Esame alla Versione Attuale **50**

A.1 Modifiche al Template 50

A.2 Errori Corretti 51

A.3 Modifiche Strutturali 51

Bibliografia **53**

Elenco delle figure

3.1 Moduli del programma e strumenti di supporto. In blu i moduli C, in arancione i kernel assembly, in verde gli script Python. . . 17

3.2 Disposizione in memoria di un dataset con $d = 13$ nella variante a 64 bit: ogni riga è completata con tre zeri fino a 16 valori, cioè 128 byte, e il blocco inizia a un indirizzo multiplo di 32. 19

3.3 A sinistra, un Ball Tree su punti del piano: la radice contiene tutti i punti, i figli A e B ne contengono una parte ciascuno, A è divisa a sua volta in due foglie. L'intorno del punto q interseca A e A2, mentre B viene esclusa senza calcolare alcuna distanza dai suoi punti. A destra, l'albero e il vettore `order`, in cui ogni nodo occupa un intervallo contiguo. 20

4.1 Una iterazione del kernel AVX e la riduzione finale delle quattro somme parziali. 26

5.1 Le due fasi dell'algoritmo: la costruzione del Ball Tree e le interrogazioni sono parallele, l'espansione dei cluster è sequenziale. 30

6.1 Tempo di esecuzione delle varianti su 20.000 punti in 16 dimensioni. 36

6.2 Speedup al crescere dei thread OpenMP. La linea tratteggiata è lo speedup ideale, la linea punteggiata il numero di core fisici. . 39

6.3 Tempo di esecuzione al crescere del numero di punti, in 16 dimensioni. 40

6.4 Tempo al variare della dimensione massima delle foglie, relativo al valore migliore di ciascun dataset. 41

6.5 Distanze calcolate dal Ball Tree, in percentuale di quelle della ricerca esaustiva, al variare della dimensione. 43

Elenco delle tabelle

3.1 Le quattro varianti prodotte da `make`. 17

3.2 Opzioni della riga di comando. 22

6.1 Ambiente delle misure. 33

6.2 Varianti a confronto su 20.000 punti in 16 dimensioni; lo speedup è rispetto alla versione d'esame. 35

6.3 Variante a 32 bit e variante a 64 bit su 20.000 punti in 16 dimensioni. 37

6.4 Varianti a confronto sul dataset dell'esame: 5000 punti uniformi in 50 dimensioni, $\varepsilon = 5$. 38

6.5 Ball Tree e ricerca esaustiva al variare della dimensione, su 20.000 punti; la colonna Distanze riporta le distanze calcolate dal Ball Tree in percentuale di quelle della ricerca esaustiva. 42

6.6 Kernel in C e kernel SIMD con i livelli di ottimizzazione `-O0` e `-O2`, su 20.000 punti in 16 dimensioni. 44

A.1 Errori della versione d'esame e relative correzioni. 51

Capitolo 1

Sommario e Introduzione

1.1 Descrizione del Progetto

Il progetto consiste nell'implementazione dell'algoritmo di clustering **DBSCAN** [1] in linguaggio C e assembly x86 e nella sua ottimizzazione su due fronti: il parallelismo a livello di dato, ottenuto con le istruzioni **SIMD** delle estensioni SSE e AVX, e il parallelismo a livello di thread, ottenuto con **OpenMP**. Il lavoro è stato svolto per il corso di Architetture e Programmazione dei Sistemi di Elaborazione, a partire dal template fornito dal docente, che stabilisce il formato dei dati di ingresso e i vincoli tecnologici: una versione a 32 bit in singola precisione con istruzioni SSE e una versione a 64 bit in doppia precisione con istruzioni AVX, ciascuna con e senza OpenMP.

Il problema

DBSCAN raggruppa i punti di un dataset in base alla densità: un punto appartiene a un cluster se nel suo intorno di raggio ε cadono abbastanza altri punti, e i cluster crescono propagandosi da un punto denso all'altro. A differenza di algoritmi come *k-means*, non richiede di conoscere il numero di cluster, riconosce cluster di forma arbitraria e isola i punti di rumore. Il suo costo è dominato dalle **interrogazioni di vicinato**: per ogni punto occorre trovare tutti i punti entro distanza ε, e una ricerca esaustiva richiede n^2 calcoli di distanza. Sono quindi tre le leve su cui agire:

1. **calcolare meno distanze**, con una struttura dati spaziale che escluda in blocco le regioni lontane dal punto interrogato: in questo progetto un *Ball Tree*;

2. **calcolare ogni distanza più in fretta**, elaborando più coordinate con una sola istruzione SIMD;

3. **calcolare più distanze contemporaneamente**, distribuendo le interrogazioni sui core del processore con OpenMP.

Soluzione adottata

Il programma legge un dataset in formato binario, esegue DBSCAN e scrive l'etichetta di ogni punto: l'indice del cluster, a partire da 0, oppure -1 per il rumore, la stessa convenzione della libreria *scikit-learn* [4]. Le scelte principali sono le seguenti:

- l'algoritmo è riformulato in **due fasi**: prima si calcolano in parallelo gli intorni di tutti i punti, poi si espandono i cluster in modo sequenziale leggendo gli intorni già calcolati;

- le interrogazioni usano un **Ball Tree**, costruito partizionando sul posto un unico vettore di indici, che esclude le sfere lontane dal punto e accetta in blocco quelle interamente contenute nell'intorno;

- le distanze sono calcolate da **kernel scritti in assembly**, SSE a 32 bit e AVX a 64 bit, su righe del dataset allineate in memoria e completate con zeri fino a un multiplo di quattro valori;

- OpenMP distribuisce le interrogazioni della prima fase e la costruzione dell'albero; il tempo è misurato come tempo reale trascorso e non come tempo di CPU.

Risultati in sintesi

Su un dataset di 20.000 punti in 16 dimensioni, con otto cluster e il 5% di rumore, la versione a 64 bit esegue il clustering in 0,48 s su un singolo thread e in 0,13 s con otto thread, contro 2,03 s della versione consegnata all'esame e

0,36 s di scikit-learn. Le etichette coincidono con quelle di scikit-learn punto per punto su tutti i dataset di prova. Il Capitolo 6 riporta le misure complete.

La versione consegnata e la versione attuale

La presente relazione descrive la versione del progetto rivista dopo l'esame. La versione consegnata è conservata nel repository con l'etichetta `exam-2024`; la revisione ne ha corretto alcuni errori, tra cui il fatto che OpenMP non venisse mai effettivamente attivato in compilazione, e ne ha ristrutturato il codice. L'Appendice A elenca le differenze tra le due versioni.

1.2 Organizzazione della Relazione in Capitoli

Il documento è articolato in otto capitoli, che procedono dall'algoritmo alle scelte implementative e si chiudono con la valutazione sperimentale. Di seguito è fornita una panoramica dei contenuti.

Capitolo 2 — L'algoritmo DBSCAN

Il capitolo richiama le definizioni del clustering basato sulla densità e l'algoritmo originale, fissa le convenzioni adottate (etichette, distanza, regola di assegnazione dei punti di confine) e introduce la formulazione in due fasi su cui si basa l'implementazione.

Capitolo 3 — Architettura del Software e Strutture Dati

Il capitolo descrive l'organizzazione in moduli, il formato dei file e la disposizione del dataset in memoria, l'array dinamico usato per gli intorni e il Ball Tree, dalla costruzione alle interrogazioni.

Capitolo 4 — Ottimizzazione mediante Istruzioni SIMD

Il capitolo presenta i kernel assembly a 32 e a 64 bit, le convenzioni di chiamata che rispettano, i vincoli di allineamento e il confronto con il codice generato dal compilatore.

Capitolo 5 — Parallelizzazione con OpenMP

Il capitolo analizza i limiti dell'approccio della versione d'esame e descrive la parallelizzazione adottata: le interrogazioni della prima fase, la costruzione del Ball Tree con i *task* e le ragioni per cui l'espansione resta sequenziale.

Capitolo 6 — Validazione e Analisi Sperimentale

Il capitolo verifica la correttezza rispetto a scikit-learn e misura l'effetto di ciascuna ottimizzazione: confronto tra le varianti, scalabilità con i thread e con il numero di punti, dimensione delle foglie del Ball Tree ed effetto della dimensionalità.

Capitolo 7 — Limiti e Sviluppi Possibili

Il capitolo discute i limiti della soluzione, a partire dalla memoria occupata dagli intorni, e le ottimizzazioni che non sono state realizzate.

Capitolo 8 — Conclusioni

Il capitolo riassume i risultati ottenuti e il contributo di ciascuna tecnica.

Capitolo 2

L'algoritmo DBSCAN

2.1 Clustering Basato sulla Densità

DBSCAN (*Density-Based Spatial Clustering of Applications with Noise*) è stato proposto da Ester, Kriegel, Sander e Xu nel 1996 [1]. L'idea è che un cluster sia una regione dello spazio in cui i punti sono più densi che nelle regioni circostanti. La densità è misurata con due parametri: il raggio ε di un intorno e il numero minimo di punti *MinPts* che l'intorno deve contenere. Dato un dataset D e una distanza dist, si definiscono:

- l'ε**-intorno** di un punto p: $N_\varepsilon(p) = \{\, q \in D \mid \mathrm{dist}(p,q) \leq \varepsilon \,\}$, che comprende p stesso;
- **punto core**: un punto p con $|N_\varepsilon(p)| \geq MinPts$;
- **direttamente raggiungibile**: q è direttamente raggiungibile per densità da p se p è un punto core e $q \in N_\varepsilon(p)$;
- **raggiungibile per densità**: q è raggiungibile da p se esiste una catena $p = p_1, \ldots, p_k = q$ in cui ogni punto è direttamente raggiungibile dal precedente;
- **connessi per densità**: p e q sono connessi se esiste un punto o da cui entrambi sono raggiungibili.

Un **cluster** è un insieme non vuoto di punti, massimale rispetto alla raggiungibilità e i cui punti sono a due a due connessi per densità. I punti di un

cluster che non sono core si dicono **punti di confine** (*border*); i punti che non appartengono ad alcun cluster sono **rumore** (*noise*). Un punto di confine può essere raggiungibile da due cluster diversi: in quel caso l'appartenenza dipende dall'ordine in cui i punti vengono visitati, come osservano gli stessi autori.

2.2 L'Algoritmo Originale

L'Algoritmo 1 riporta lo schema del lavoro originale. Si scorrono i punti e, da ogni punto core non ancora classificato, si fa crescere un nuovo cluster con la procedura ExpandCluster, che usa un insieme di *seed* come frontiera di visita: a ogni passo si estrae un seed, si calcola il suo intorno e, se è un punto core, si aggiungono i vicini non ancora visitati.

Algoritmo 1 DBSCAN nella formulazione di Ester et al. [1]

Input: dataset D, raggio ε, soglia *MinPts*

```
1:  c ← 0; tutti i punti sono UNCLASSIFIED
2:  per ogni p ∈ D esegui
3:      se p è UNCLASSIFIED allora
4:          S ← REGIONQUERY(p, ε)
5:          se |S| < MinPts allora
6:              p ← NOISE                ▷ potrà diventare un punto di confine
7:          altrimenti
8:              assegna c a tutti i punti di S; togli p da S
9:              finché S ≠ ∅ esegui
10:                 estrai q da S; R ← REGIONQUERY(q, ε)
11:                 se |R| ≥ MinPts allora
12:                     per ogni r ∈ R UNCLASSIFIED o NOISE esegui
13:                         se r è UNCLASSIFIED, aggiungilo a S; assegna c a r
14:                     fine per
15:                 fine se
16:             fine finché
17:             c ← c + 1
18:         fine se
19:     fine se
20: fine per
```

Ogni punto viene interrogato una sola volta, a parte i punti di rumore che rientrano tra i seed di un cluster, quindi il costo complessivo è $O(n \cdot Q)$, dove Q è il costo di una RegionQuery. Con una ricerca esaustiva $Q = O(n\,d)$ e l'algoritmo diventa quadratico nel numero di punti. Con un indice spaziale Q scende, nei casi favorevoli, a un costo logaritmico più la dimensione dell'intorno; Schubert et al. [2] mostrano però che questo vale solo se ε è piccolo rispetto alla scala dei dati, perché con intorni grandi la dimensione stessa dei risultati rende il costo quadratico.

2.3 Convenzioni Adottate

Etichette

Il risultato è un vettore di interi con un'etichetta per punto: l'indice del cluster, numerato da 0 nell'ordine in cui i cluster vengono scoperti, oppure -1 (`NOISE`) per il rumore. Durante l'elaborazione il valore -2 (`UNCLASSIFIED`) indica un punto non ancora raggiunto. È la stessa convenzione di scikit-learn [4], il che permette di confrontare i risultati direttamente.

Il parametro MinPts

MinPts conta anche il punto stesso, come nella definizione di ε-intorno e come il parametro `min_samples` di scikit-learn. Internamente le interrogazioni restituiscono l'intorno *senza* il punto interrogato, che non servirebbe in alcun passo successivo, e la condizione di punto core diventa quindi $|N_\varepsilon(p) \setminus \{p\}| + 1 \geq$ *MinPts*.

Distanza euclidea e confronti al quadrato

La distanza è quella euclidea, indotta dalla norma 2, in accordo con scikit-learn. Poiché la radice quadrata è monotona crescente, $\mathrm{dist}(p, q) \leq \varepsilon$ equivale a $\mathrm{dist}(p, q)^2 \leq \varepsilon^2$: i kernel calcolano quindi la **distanza al quadrato** e la radice viene estratta solo dove serve una distanza vera, cioè nel Ball Tree per i raggi delle sfere e per i test di esclusione. Una norma diversa, per esempio la norma 1 (distanza di Manhattan), individuerebbe intorni diversi e quindi cluster diversi: i risultati non sarebbero più confrontabili con quelli di riferimento.

Assegnazione dei punti di confine

Quando un punto di confine è raggiungibile da più cluster, l'implementazione lo assegna al **primo** cluster che lo raggiunge, come scikit-learn. L'Algoritmo 1, invece, assegna al nuovo cluster tutti i punti dell'intorno iniziale, anche quelli già attribuiti a un cluster precedente. Entrambe le scelte sono legittime, ma la prima rende il risultato identico a quello di scikit-learn e quindi verificabile in modo esatto.

2.4 Formulazione in Due Fasi

Nell'Algoritmo 1 le interrogazioni sono intrecciate con l'espansione: quale punto interrogare dopo dipende dal risultato delle interrogazioni precedenti. Questo lascia poco spazio al parallelismo, che nella versione d'esame era stato cercato all'interno delle singole interrogazioni (Capitolo 5). L'implementazione separa invece il lavoro in due fasi (Algoritmo 2):

1. **Fase 1, intorni**: si calcola l'intorno di ogni punto e si stabilisce se è un punto core. Le interrogazioni sono indipendenti tra loro e possono essere eseguite in parallelo. Si conservano solo gli intorni dei punti core, gli unici che l'espansione leggerà.

2. **Fase 2, espansione**: i cluster crescono con una visita in profondità del grafo che ha per nodi i punti e per archi le relazioni di raggiungibilità diretta, usando una pila. Nessuna distanza viene più calcolata.

Algoritmo 2 DBSCAN in due fasi, come implementato in `src/dbscan.c`

Input: dataset D di n punti, raggio ε, soglia *MinPts*

Output: etichette $L[0..n)$

```
1: per ogni p ∈ D esegui                                   ▷ Fase 1, in parallelo
2:     N[p] ← RegionQuery(p, ε) \ {p}
3:     core[p] ← |N[p]| + 1 ≥ MinPts
4:     se non core[p] allora
5:         scarta N[p]
6:     fine se
7: fine per
8: L ← Unclassified; c ← 0                                 ▷ Fase 2, sequenziale
9: per p ← 0 to n − 1 esegui
10:    se L[p] = Unclassified e core[p] allora
11:        L[p] ← c; inserisci p nella pila
12:        finché la pila non è vuota esegui
13:            q ← estrai dalla pila                       ▷ sempre un punto core
14:            per ogni r ∈ N[q] con L[r] = Unclassified esegui
15:                L[r] ← c
16:                se core[r] allora
17:                    inserisci r nella pila
18:                fine se
19:            fine per
20:        fine finché
21:        c ← c + 1
22:    fine se
23: fine per
24: i punti rimasti Unclassified diventano Noise
```

La seconda fase produce gli stessi cluster dell'algoritmo originale: un cluster nasce dal primo punto core non ancora etichettato, nello stesso ordine di scansione, e contiene tutti i punti raggiungibili da esso, perché la visita prosegue solo attraverso i punti core ed etichetta ogni loro vicino. Anche il numero di interrogazioni resta lo stesso, una per punto. Il prezzo è la memoria: gli intorni

dei punti core restano in memoria fino alla fine, per un totale proporzionale alla somma delle loro dimensioni. È la stessa scelta di scikit-learn, che calcola tutti gli intorni prima dell'espansione, e se ne discutono i limiti nel Capitolo 7.

Capitolo 3

Architettura del Software e Strutture Dati

3.1 Organizzazione dei Moduli

Il codice è un unico insieme di sorgenti C11 e assembly NASM, organizzato in moduli con responsabilità distinte (Figura 3.1). Le quattro varianti richieste dal corso non sono più quattro copie del programma, come nella versione d'esame, ma quattro compilazioni degli stessi sorgenti, prodotte dal `Makefile` (Tabella 3.1): la precisione dipende dall'architettura di destinazione, attraverso il tipo `real_t` definito in `common.h`, e OpenMP si attiva con l'opzione `-fopenmp` del compilatore. Senza di essa le direttive `#pragma omp` vengono ignorate e il programma resta sequenziale.

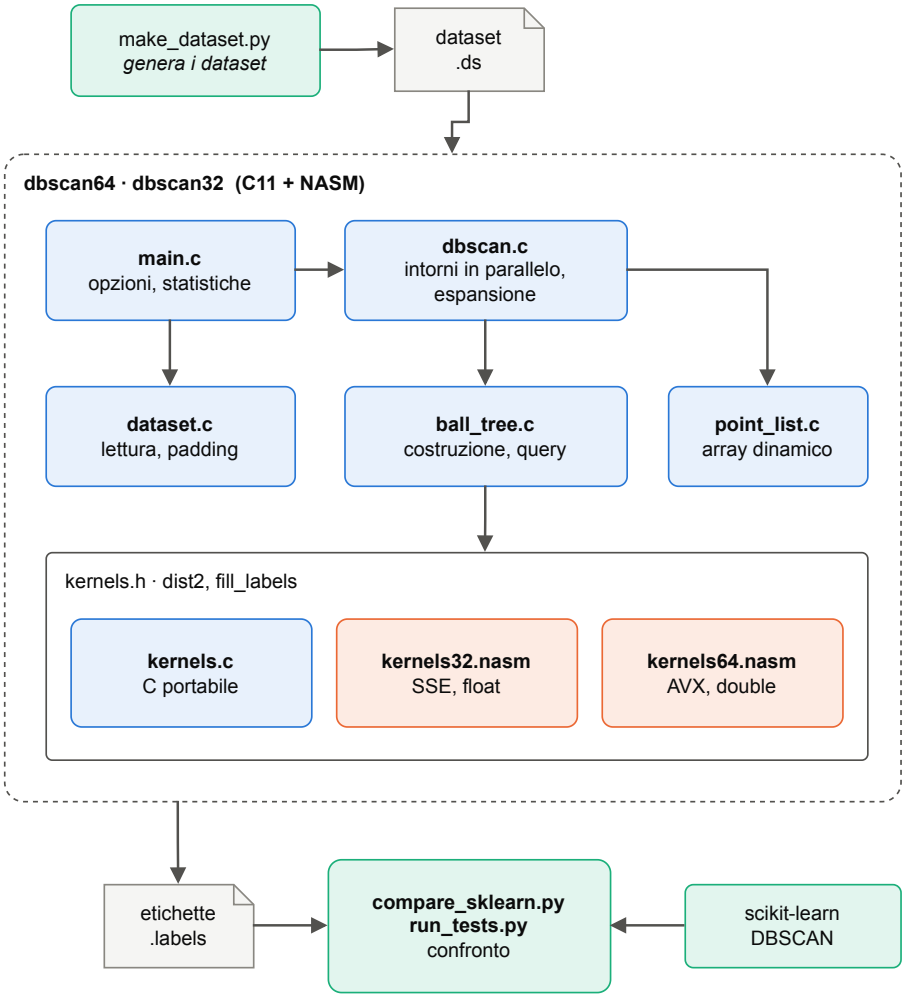


Figura 3.1: Moduli del programma e strumenti di supporto. In blu i moduli C, in arancione i kernel assembly, in verde gli script Python.

Eseguibile	**Architettura**	**Precisione e kernel**	**OpenMP**
`dbscan32`	x86-32	`float`, SSE	no
`dbscan32-omp`	x86-32	`float`, SSE	sì
`dbscan64`	x86-64	`double`, AVX	no
`dbscan64-omp`	x86-64	`double`, AVX	sì

Tabella 3.1: Le quattro varianti prodotte da `make`.

I moduli sono i seguenti:

- `main.c`: interpreta la riga di comando, carica il dataset, invoca l'algoritmo e stampa le statistiche, anche in formato JSON per gli script di misura;

- `dataset.c`: legge i file, converte i valori in `real_t`, completa le righe con zeri e le allinea in memoria; scrive le etichette;
- `dbscan.c`: l'algoritmo in due fasi della Sezione 2.4 e la ricerca esaustiva;
- `ball_tree.c`: costruzione e interrogazione del Ball Tree;
- `point_list.c`: l'array dinamico di indici usato per gli intorni e come pila;
- `kernels.h`: l'interfaccia dei due kernel, la distanza al quadrato `dist2` e il riempimento delle etichette `fill_labels`, con una versione in C (`kernels.c`) e una in assembly per ciascuna architettura (`asm/kernels32.nasm` e `asm/kernels64.nasm`). La versione da usare è scelta all'avvio tramite un puntatore a funzione, così che le due si possano confrontare con lo stesso eseguibile.

Accanto al programma, alcuni script Python completano il progetto:

- `make_dataset.py` genera i dataset;
- `compare_sklearn.py` e `tests/run_tests.py` confrontano le etichette con quelle di scikit-learn;
- `benchmark.py` esegue le misure del Capitolo 6, e `report_data.py` ne ricava tabelle e grafici.

3.2 Formato dei File e Disposizione in Memoria

Formato dei file

Il formato è quello del template del corso: due interi a 32 bit, il numero di colonne e il numero di righe, seguiti dai valori della matrice riga per riga (*row-major order*). I valori sono numeri in virgola mobile a 32 o a 64 bit; il programma deduce la precisione dalla dimensione del file e converte i valori nel tipo della propria variante, così lo stesso dataset si può usare con entrambe le architetture. Le etichette sono scritte nello stesso formato, come matrice di una colonna di interi a 32 bit.

Padding e allineamento

Le istruzioni SIMD lavorano su blocchi di quattro valori: quattro `float` in un registro XMM a 128 bit o quattro `double` in un registro YMM a 256 bit. Le istruzioni di caricamento allineato (`movaps`, `vmovapd`) richiedono inoltre che l'indirizzo sia multiplo della dimensione del registro, 16 o 32 byte, e generano un'eccezione in caso contrario. Il dataset viene quindi memorizzato come un unico blocco allineato, in cui ogni riga ha un numero di valori, detto *stride*, pari alla dimensione d arrotondata al multiplo di quattro successivo; le colonne aggiunte valgono zero e non alterano le distanze. Poiché la lunghezza di una riga in byte è un multiplo della dimensione di un registro, tutte le righe risultano allineate (Figura 3.2), e i kernel possono elaborare ogni riga a blocchi interi senza gestire un residuo.

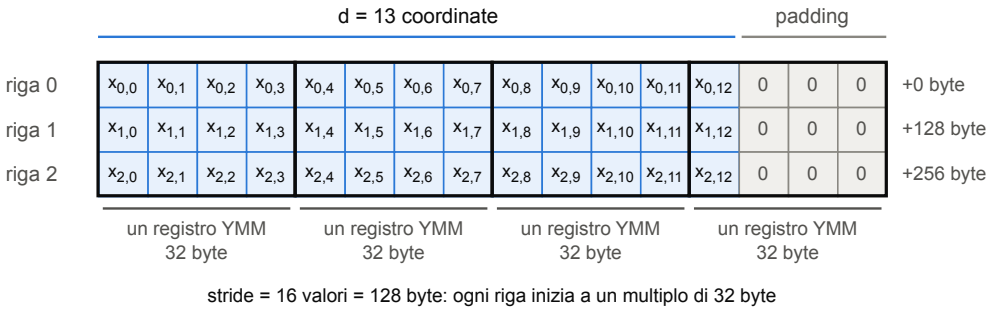


Figura 3.2: Disposizione in memoria di un dataset con $d = 13$ nella variante a 64 bit: ogni riga è completata con tre zeri fino a 16 valori, cioè 128 byte, e il blocco inizia a un indirizzo multiplo di 32.

La versione d'esame otteneva lo stesso risultato con la funzione `adjust_matrix`, che però allocava una seconda matrice senza liberare la prima; ora il completamento avviene durante la lettura, in un'unica allocazione.

3.3 L'Array Dinamico PointList

Gli intorni hanno dimensioni imprevedibili, da zero a $n - 1$ punti. `PointList` è un array di indici che raddoppia la propria capacità quando si riempie: il costo di un inserimento è costante in media, e la memoria sprecata è al più pari a quella usata. La stessa struttura serve da pila nella seconda fase.

Nella versione d'esame questo meccanismo, insieme alla deallocazione esplicita delle liste non più necessarie, era presentato come un sistema di *garbage collection*; si tratta in realtà di gestione manuale della memoria con un array a crescita geometrica. Nella versione attuale ogni thread riusa la stessa `PointList` per tutte le proprie interrogazioni, svuotandola di volta in volta, e copia il risultato in un vettore della dimensione esatta solo se il punto è core: le allocazioni si riducono a una per punto core.

3.4 Il Ball Tree

Un Ball Tree [3] è un albero binario in cui ogni nodo rappresenta una ipersfera, detta *ball*, che contiene un sottoinsieme dei punti: il centro è il centroide dei punti e il raggio è la distanza dal centro al punto più lontano. I due figli di un nodo si spartiscono i suoi punti; le foglie contengono al più *leaf_size* punti. Rispetto a un *k-d tree*, che divide lo spazio con iperpiani paralleli agli assi, le sfere si adattano meglio alla distribuzione dei dati quando la dimensione cresce.

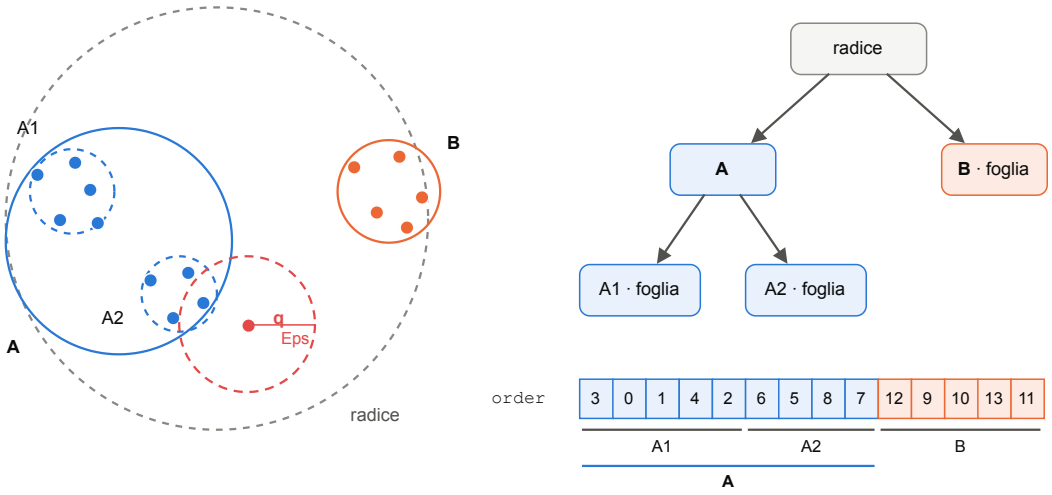


Figura 3.3: A sinistra, un Ball Tree su punti del piano: la radice contiene tutti i punti, i figli A e B ne contengono una parte ciascuno, A è divisa a sua volta in due foglie. L'intorno del punto q interseca A e A2, mentre B viene esclusa senza calcolare alcuna distanza dai suoi punti. A destra, l'albero e il vettore `order`, in cui ogni nodo occupa un intervallo contiguo.

Costruzione

La costruzione parte da un nodo che contiene tutti i punti e lo divide ricorsivamente. Per dividere un nodo si scelgono due punti lontani tra loro: p_1, il più lontano dal centroide, e p_2, il più lontano da p_1. Ogni punto va poi con il più vicino dei due. Il criterio costa due scansioni lineari del nodo e tende a separare i gruppi di punti densi, perché p_1 e p_2 cadono agli estremi opposti della distribuzione.

L'albero non memorizza liste di punti per nodo. Tutti i nodi condividono un unico vettore di indici, `order`, e la suddivisione ne riordina sul posto l'intervallo con uno schema a due indici simile a quello del *quicksort*: ogni nodo occupa un intervallo contiguo `[start, end)` di `order`, i figli ne occupano le due metà (Figura 3.3). Nella versione d'esame ogni nodo allocava invece due nuove liste di punti e liberava quella del padre, con un gran numero di allocazioni. Al termine della costruzione le righe del dataset vengono copiate nell'ordine di `order`: così la scansione di una foglia legge memoria contigua invece di saltare tra righe sparse. La copia raddoppia la memoria occupata dal dataset ma riduce il tempo di esecuzione del 20–40%, come emerso durante lo sviluppo.

Un nodo diventa foglia quando ha al più *leaf_size* punti oppure quando tutti i suoi punti coincidono, perché in quel caso il raggio è nullo e nessuna divisione è possibile. La versione d'esame non gestiva questo caso: con più di *leaf_size* punti identici, tutti finivano nello stesso figlio e la ricorsione non terminava. È presente anche una salvaguardia per il caso, dovuto soltanto agli arrotondamenti, in cui uno dei due figli risulti vuoto.

Interrogazione

Per trovare l'intorno di un punto x si visita l'albero dalla radice. Per ogni nodo di centro c e raggio r si calcola $\delta = \mathrm{dist}(x, c)$ e, per la disuguaglianza triangolare, si distinguono tre casi:

1. se $\delta > r + \varepsilon$, la sfera del nodo e l'intorno di x sono disgiunti: il nodo e tutto il suo sottoalbero vengono esclusi;
2. se $\delta + r \leq \varepsilon$, la sfera è interamente contenuta nell'intorno: tutti i punti del nodo vengono accettati senza calcolarne le distanze;

3. altrimenti si prosegue sui figli, oppure, in una foglia, si calcola la distanza di ogni punto.

Il secondo caso non era presente nella versione d'esame ed è decisivo quando ε è grande rispetto alla scala dei dati, come nel dataset usato per l'esame: la Sezione 6.5 mostra che riduce le distanze calcolate da 25 milioni a circa cinquemila. Le distanze dai centri sono le uniche per cui si estrae la radice quadrata, perché i test confrontano somme di distanze; i centroidi sono allocati allineati e completati come le righe del dataset, quindi anche queste distanze usano i kernel SIMD.

3.5 Interfaccia a Riga di Comando

La Tabella 3.2 riassume le opzioni del programma. Rispetto al template sono state eliminate le opzioni `-k` e `-labels`, ereditate da un altro progetto del corso e ignorate dall'algoritmo, e sono state aggiunte quelle che selezionano l'indice, i kernel e il numero di thread: le varianti confrontate nel Capitolo 6 si ottengono tutte dallo stesso eseguibile, senza modificare il codice.

Opzione	**Significato**
`-i`, `-input FILE`	dataset da elaborare
`-e`, `-eps EPS`	raggio ε dell'intorno
`-m`, `-min-pts N`	soglia *MinPts*, punto stesso incluso
`-o`, `-output FILE`	file in cui scrivere le etichette
`-index KIND`	`balltree` (predefinito) oppure `linear`, la ricerca esaustiva
`-leaf-size N`	punti massimi in una foglia del Ball Tree (predefinito 32)
`-no-simd`	usa i kernel in C invece di quelli SSE/AVX
`-t`, `-threads N`	numero di thread OpenMP (predefinito: tutti i core logici)
`-json`	stampa risultati e tempi come una riga JSON
`-q`, `-quiet`	non stampa nulla oltre agli errori e alla riga JSON

Tabella 3.2: Opzioni della riga di comando.

Capitolo 4

Ottimizzazione mediante Istruzioni SIMD

4.1 Dove Intervenire

Quasi tutto il tempo di esecuzione è speso nella prima fase, e al suo interno nel calcolo delle distanze: nelle misure del Capitolo 6 la costruzione del Ball Tree e l'espansione dei cluster pesano insieme meno del 2% del totale. Il candidato naturale per le istruzioni SIMD (*Single Instruction, Multiple Data*) è quindi il calcolo della distanza al quadrato tra due punti, che applica la stessa sequenza di operazioni — sottrazione, quadrato, somma — a tutte le coordinate. Il secondo kernel, il riempimento del vettore delle etichette con un valore costante, è eseguito una sola volta per esecuzione e ha un peso trascurabile; è mantenuto perché era parte dell'esercizio del corso e mostra le istruzioni su dati interi.

Entrambi i kernel sono scritti in assembly NASM, uno per architettura, e dichiarati in `kernels.h` con puntatori e dimensioni come parametri:

```
real_t dist2_simd(const real_t *a, const real_t *b, int d);
void fill_labels_simd(int *labels, int n, int value);
```

Nella versione d'esame le funzioni assembly ricevevano invece il puntatore alla struttura dei parametri e ne leggevano i campi a spiazzamenti fissi, per esempio `[rdi+40]` per la dimensione: una modifica alla struttura bastava a rompere il codice assembly senza alcun errore di compilazione. Con i puntatori, inoltre, lo stesso kernel serve sia per la distanza tra due righe del dataset sia per quella tra un punto e il centro di una sfera del Ball Tree.

4.2 Il Kernel a 32 Bit con SSE

Nella variante a 32 bit i valori sono `float` e un registro XMM ne contiene quattro. Il Listato 4.1 riporta il kernel della distanza.

```
dist2_simd:
        mov     eax, [esp+4]            ; a
        mov     ecx, [esp+8]            ; b
        mov     edx, [esp+12]           ; d
        shl     edx, 2                  ; lunghezza della riga in byte
        add     eax, edx                ; puntatori alla fine delle righe,
        add     ecx, edx                ; percorse con uno spiazzamento
    negativo
        neg     edx
        xorps   xmm0, xmm0              ; 4 somme parziali
.loop:
        movaps  xmm1, [eax+edx]         ; 4 coordinate di a
        subps   xmm1, [ecx+edx]         ; meno 4 coordinate di b
        mulps   xmm1, xmm1
        addps   xmm0, xmm1
        add     edx, 16
        jnz     .loop
        haddps  xmm0, xmm0              ; somma orizzontale (SSE3)
        haddps  xmm0, xmm0
        sub     esp, 4                  ; st(0) si carica solo dalla
    memoria
        movss   [esp], xmm0
        fld     dword [esp]
        add     esp, 4
        ret
```

Listato 4.1: Distanza al quadrato con SSE (`src/asm/kernels32.nasm`).

Alcune scelte meritano un commento:

- **Ciclo a spiazzamento negativo.** I puntatori sono spostati alla fine delle righe e il ciclo le percorre con un indice negativo che cresce fino a zero: l'istruzione `add` che aggiorna l'indice imposta anche il flag di zero, e il ciclo non ha bisogno di un confronto separato.
- **Nessun residuo.** Grazie al padding (Sezione 3.2) la dimensione è sempre un multiplo di quattro e le righe sono allineate, quindi il ciclo lavora solo su blocchi interi con caricamenti allineati.
- **Somme parziali.** L'accumulatore contiene quattro somme parziali indipendenti, ridotte a una sola alla fine con due `haddps`.
- **Valore di ritorno.** La convenzione di chiamata *cdecl* a 32 bit [6] prevede che una funzione restituisca i valori in virgola mobile in cima allo stack

del coprocessore x87, `st(0)`. Non è quindi una scelta, come riportava la relazione d'esame, ma un obbligo. Poiché `st(0)` si può caricare solo dalla memoria, il valore passa per una cella sullo stack del thread. La versione d'esame usava invece una variabile globale: due thread che eseguono il kernel contemporaneamente scrivono nella stessa cella, e uno dei due può restituire la distanza calcolata dall'altro.

- **Registri.** La convenzione impone di preservare `EBX`, `ESI`, `EDI` e `EBP`; il kernel usa solo `EAX`, `ECX`, `EDX` e registri XMM, che il chiamante non si aspetta di ritrovare intatti, e quindi non deve salvare nulla.

4.3 Il Kernel a 64 Bit con AVX

Nella variante a 64 bit i valori sono `double` e i registri YMM a 256 bit dell'estensione AVX ne contengono quattro, così che la struttura del ciclo resta la stessa (Listato 4.2). La convenzione di chiamata è quella System V AMD64 [6]: i primi argomenti interi arrivano in `RDI`, `RSI` e `RDX`, il risultato in virgola mobile si restituisce in `XMM0`.

```
dist2_simd:
        movsxd  rdx, edx
        shl     rdx, 3                  ; lunghezza della riga in byte
        add     rdi, rdx
        add     rsi, rdx
        neg     rdx
        vxorpd  ymm0, ymm0, ymm0        ; 4 somme parziali
.loop:
        vmovapd ymm1, [rdi+rdx]
        vsubpd  ymm1, ymm1, [rsi+rdx]
        vmulpd  ymm1, ymm1, ymm1
        vaddpd  ymm0, ymm0, ymm1
        add     rdx, 32
        jnz     .loop
        vextractf128 xmm1, ymm0, 1      ; somma la meta' alta alla bassa
        vaddpd  xmm0, xmm0, xmm1
        vhaddpd xmm0, xmm0, xmm0        ; e le due corsie rimaste
        vzeroupper
        ret
```

Listato 4.2: Distanza al quadrato con AVX (`src/asm/kernels64.nasm`).

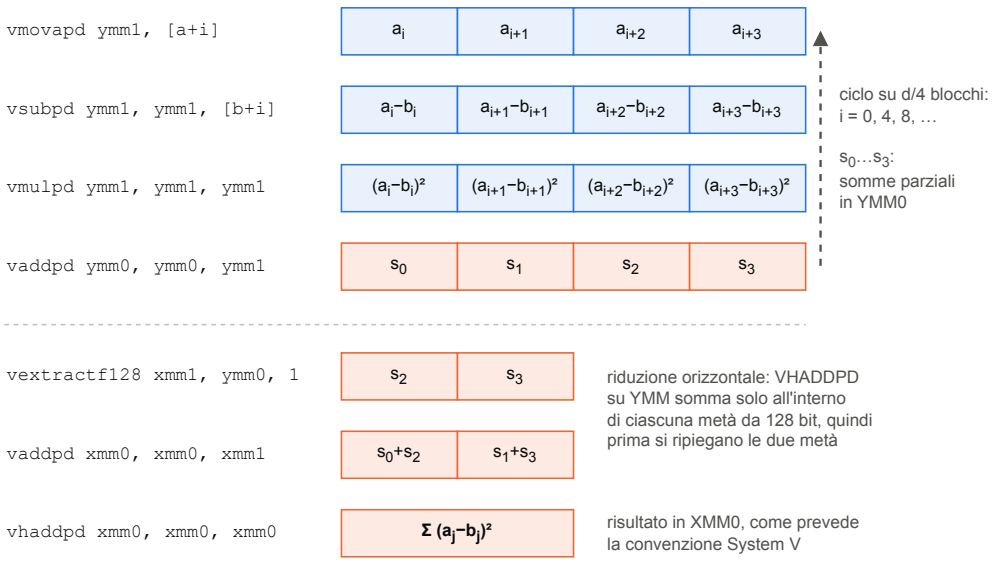


Figura 4.1: Una iterazione del kernel AVX e la riduzione finale delle quattro somme parziali.

La riduzione finale (Figura 4.1) è diversa da quella a 32 bit perché `vhaddpd` su registri YMM somma le coppie di valori all'interno di ciascuna metà da 128 bit, senza mai combinare le due metà: occorre prima estrarre la metà alta con `vextractf128` e sommarla a quella bassa. Prima di tornare al chiamante, `vzeroupper` azzera la metà alta dei registri YMM. Sui processori Intel [5], eseguire istruzioni SSE non codificate VEX mentre le metà alte sono «sporche» comporta una penalità di transizione, e il codice C compilato da GCC nel chiamante può contenerne. Per lo stesso motivo il kernel usa solo istruzioni con codifica VEX, mentre il kernel d'esame alternava istruzioni AVX e SSE (`movsd`, `subsd`, `sqrtsd`) nello stesso ciclo e non eseguiva `vzeroupper`.

Anche qui il kernel usa solo registri che la convenzione non chiede di preservare. Il kernel d'esame salvava invece quattro registri con `push rax, rbx, rcx, rdi` e li ripristinava con `pop` nello stesso ordine, anziché in ordine inverso: al ritorno `RBX`, che il chiamante si aspetta invariato, conteneva il valore di `RCX`. Il programma funzionava solo perché il codice generato con `-O0` non usava `RBX` in quel punto. Nella stessa versione, la macro `popaq` fornita con il template ripristinava anche i registri YMM, sovrascrivendo il risultato appena calcolato in `XMM0`: è questo il comportamento osservato nella relazione d'esame e attribuito, erroneamente, al processore.

4.4 Il Riempimento delle Etichette

`fill_labels_simd` replica un intero in tutte le corsie di un registro e lo scrive a blocchi: quattro etichette per istruzione con SSE (`pshufd` e `movdqa`), otto con AVX. L'estensione AVX non dispone di rimescolamenti su interi a 256 bit, quindi il valore viene replicato in un registro XMM e copiato nella metà alta con `vinsertf128`. Un breve ciclo scalare scrive le etichette residue quando n non è multiplo della larghezza del registro. Rispetto alla versione d'esame, il valore da scrivere è un parametro invece di una costante in memoria, e le condizioni di uscita sono corrette per ogni n, compresi i valori minori di quattro.

4.5 Confronto con il Codice del Compilatore

Il template del corso prescriveva la compilazione con `-O0`, per rendere evidente il contributo del codice assembly. La versione attuale è compilata con `-O2`, e a questo livello GCC vettorizza da sé il ciclo della versione C del kernel. Il codice generato (Listato 4.3) mostra però che la vettorizzazione è solo parziale.

```
.L4:    vmovupd     (%rdi,%rax), %ymm1
        vsubpd      (%rsi,%rax), %ymm1, %ymm1
        addq        $32, %rax
        vmulpd      %ymm1, %ymm1, %ymm1
        vaddsd      %xmm1, %xmm0, %xmm0
        vunpckhpd   %xmm1, %xmm1, %xmm2
        vextractf128 $0x1, %ymm1, %xmm1
        vaddsd      %xmm2, %xmm0, %xmm0
        vaddsd      %xmm1, %xmm0, %xmm0
        vunpckhpd   %xmm1, %xmm1, %xmm1
        vaddsd      %xmm1, %xmm0, %xmm0
        cmpq        %rcx, %rax
        jne         .L4
```

Listato 4.3: Ciclo di `dist2_c` generato da GCC 16 con `-O2 -mavx` (sintassi AT&T).

Sottrazioni e quadrati sono calcolati quattro alla volta, ma i quadrati vengono poi sommati all'accumulatore uno per volta, in ordine, con quattro `vaddsd` dipendenti l'una dall'altra. La somma in virgola mobile non è associativa, e senza un permesso esplicito (`-ffast-math` o `-fassociative-math`) il compilatore non può cambiare l'ordine delle addizioni. Il kernel scritto a mano lo cambia deliberatamente, mantenendo quattro somme parziali indipendenti, e completa un'iterazione con una sola somma vettoriale invece di una catena di

quattro. Il risultato può differire da quello sequenziale nelle ultime cifre, ma i test del Capitolo 6 verificano che le etichette non cambiano. La Sezione 6.10 misura il vantaggio che resta al codice scritto a mano con entrambi i livelli di ottimizzazione.

Capitolo 5

Parallelizzazione con OpenMP

5.1 L'Approccio della Versione d'Esame

OpenMP [7] permette di parallelizzare un programma C annotandolo con direttive `#pragma omp`, che il compilatore traduce in codice multi-thread quando si compila con l'opzione `-fopenmp`. La versione d'esame manteneva l'algoritmo originale, con le interrogazioni intrecciate all'espansione, e cercava il parallelismo all'interno della singola interrogazione:

- la visita ricorsiva del Ball Tree apriva una regione `parallel sections` per esplorare in parallelo i due figli di ogni nodo;
- la scansione di una foglia era un ciclo `parallel for`, con una sezione `critical` attorno a ogni inserimento di un punto nel risultato;
- l'assegnazione dell'etichetta ai punti di un intorno era a sua volta un ciclo `parallel for`.

L'approccio aveva tre problemi. Il primo è che non è mai stato eseguito: gli script di compilazione della cartella OpenMP non passavano l'opzione `-fopenmp`, il compilatore ignorava le direttive, e il binario a 32 bit «con OpenMP» risultava identico byte per byte a quello senza. Il secondo è la granularità: una interrogazione dura pochi microsecondi, meno del costo di creare e sincronizzare un gruppo di thread, e ogni livello della ricorsione ne apriva uno nuovo; la sezione critica serializzava poi proprio gli inserimenti. Il terzo è la misura: il tempo era calcolato con `clock()`, che restituisce il tempo di CPU del processo,

cioè la somma dei tempi di tutti i thread, e che con più thread cresce anche quando il programma diventa più veloce. La relazione d'esame dichiarava infatti di non poter misurare il beneficio di OpenMP.

5.2 Parallelizzazione della Prima Fase

La formulazione in due fasi (Sezione 2.4) sposta il parallelismo al livello più alto possibile: le n interrogazioni della prima fase sono indipendenti e vengono distribuite tra i thread con un unico ciclo parallelo (Figura 5.1).

```
#pragma omp parallel reduction(+ : evals, kept, core_points)
{
    PointList found; // riusata da tutte le interrogazioni del thread
    point_list_init(&found, 256);
#pragma omp for schedule(dynamic, 64)
    for (int p = 0; p < n; p++) {
        point_list_clear(&found);
        evals += ball_tree_query(&tree, ds, p, eps, dist2, &found);
        is_core[p] = found.size + 1 >= min_pts;
        if (is_core[p])
            neighbours[p] = copia di found.items;
    }
    point_list_free(&found);
}
```

Listato 5.1: La prima fase in `src/dbscan.c`, semplificata.

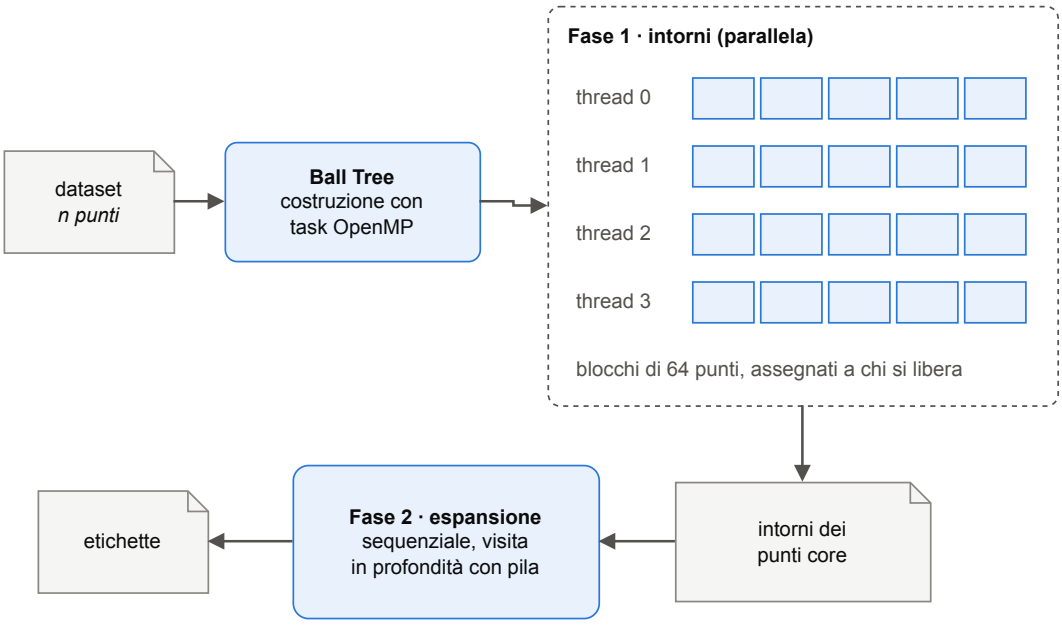


Figura 5.1: Le due fasi dell'algoritmo: la costruzione del Ball Tree e le interrogazioni sono parallele, l'espansione dei cluster è sequenziale.

Le scelte principali sono tre:

- **nessuna sincronizzazione nel ciclo**: ogni iterazione scrive solo nelle celle di indice p dei vettori `is_core`, `count` e `neighbours`, e ogni thread ha la propria lista temporanea. I contatori statistici sono sommati con una clausola `reduction`;
- **assegnazione dinamica**: il costo di un'interrogazione varia molto da punto a punto, perché un punto al centro di un cluster ha molti vicini e un punto isolato quasi nessuno. Con `schedule(dynamic, 64)` i thread prelevano blocchi di 64 punti man mano che si liberano, invece di ricevere in anticipo una porzione fissa, e il carico si bilancia da sé. Il blocco di 64 punti rende trascurabile il costo della distribuzione;
- **ordine indipendente dai thread**: ogni intorno contiene sempre gli stessi punti, e la seconda fase li legge in ordine di indice. Il risultato è quindi identico con qualunque numero di thread, come verificano i test.

5.3 Costruzione Parallela del Ball Tree

La costruzione del Ball Tree è ricorsiva e i due sottoalberi di un nodo sono indipendenti, perché occupano intervalli disgiunti del vettore `order`. Ogni chiamata ricorsiva diventa quindi un *task* OpenMP, creato all'interno di una regione parallela da un solo thread (`single`) ed eseguito da un thread qualunque del gruppo; `taskwait` attende i due figli prima di restituire il nodo. Sotto i 4096 punti la clausola `if` esegue il sottoalbero direttamente, perché un task così piccolo costerebbe più di quanto fa risparmiare. Il guadagno è limitato dal fatto che la suddivisione della radice, che esamina tutti i punti, è necessariamente eseguita da un solo thread; la costruzione pesa comunque circa l'1% del tempo totale.

5.4 Perché l'Espansione Resta Sequenziale

La seconda fase non calcola distanze: visita un grafo i cui archi sono già noti, con un costo proporzionale alla somma delle dimensioni degli intorni. Nelle misure pesa tra lo 0,2% e l'1% del tempo totale, secondo il numero di thread,

quindi parallelizzarla porterebbe un beneficio trascurabile. Sarebbe anche difficile farlo senza cambiare il risultato: due thread che espandono cluster diversi possono contendersi un punto di confine, e l'assegnazione al primo cluster che lo raggiunge richiede un ordine di visita definito. Una versione parallela dovrebbe calcolare le componenti connesse dei punti core con una struttura *union-find* e assegnare i punti di confine in un passo successivo (Capitolo 7).

5.5 Misura del Tempo

Il programma misura il tempo reale trascorso, con la funzione `clock_gettime` e l'orologio monotono del sistema, separatamente per la costruzione dell'indice, la prima e la seconda fase. Il tempo riportato comprende solo il clustering e non la lettura del file, come nella versione d'esame, così che le due siano confrontabili. Per la versione d'esame, che è sequenziale, il tempo di CPU misurato da `clock()` coincide in pratica con il tempo reale.

Capitolo 6

Validazione e Analisi Sperimentale

6.1 Ambiente e Metodologia

Tutte le misure sono state eseguite sulla macchina descritta nella Tabella 6.1 e sono riproducibili con `make bench`, che genera i dataset, esegue gli esperimenti e salva i risultati grezzi nella cartella `results`; lo script `report_data.py` ne ricava le tabelle e i grafici di questo capitolo.

Processore	Intel(R) Core(TM) i5-8300H CPU @ 2.30GHz (4 core, 8 thread)
Sistema operativo	Fedora Linux 44 (KDE Plasma Desktop Edition), Linux 7.2.5-200.fc44.x86_64
Compilatore	gcc (GCC) 16.2.1 20260819 (Red Hat 16.2.1-2)
Assemblatore	NASM version 2.16.03
Riferimento	Python 3.12.14, NumPy 2.5.3, scikit-learn 1.9.1
Data delle misure	2026-09-24

Tabella 6.1: Ambiente delle misure.

Dataset

Salvo dove indicato, i dataset sono generati con la funzione `make_blobs` di scikit-learn: otto cluster gaussiani con deviazione standard unitaria, a cui si

aggiunge un 5% di punti distribuiti uniformemente nel riquadro che li contiene, a fare da rumore. *MinPts* vale 5 e ε è scelto con l'euristica del lavoro originale [1], basata sulla distanza di ogni punto dal suo quarto vicino: si prende il valore che lascia il 90% dei punti al di sotto, così che circa il 90% dei punti sia core. Il dataset usato per l'esame, invece, contiene 5000 punti distribuiti uniformemente nell'ipercubo unitario a 50 dimensioni, con $\varepsilon = 5$.

Misure

Ogni valore è la mediana di cinque esecuzioni. Il tempo è quello misurato dal programma per il solo clustering, dalla costruzione dell'indice alla fine dell'espansione, con la lettura del file esclusa; per scikit-learn è il tempo della chiamata a `fit`. La macchina è un portatile, con frequenza del processore variabile in base al carico e alla temperatura: le misure ripetute differiscono tipicamente di qualche punto percentuale, e minimo e massimo di ogni serie sono riportati nei file dei risultati.

6.2 Correttezza

Lo script `tests/run_tests.py`, eseguito da `make test` e dall'integrazione continua a ogni modifica, confronta le etichette prodotte da ogni eseguibile con quelle di scikit-learn su nove casi:

- tre dataset con cluster in 2, 13 e 64 dimensioni, di cui la seconda non multipla di quattro, per verificare il padding;
- il dataset uniforme dell'esame con $\varepsilon = 5$, un unico cluster, e con $\varepsilon = 1{,}2$, tutto rumore;
- 300 punti identici, il caso che nella versione d'esame causava la ricorsione infinita;
- $MinPts = 1$, in cui ogni punto è core;
- dataset di uno e di tre punti, più piccoli di un registro SIMD.

Ogni caso è eseguito con la ricerca esaustiva e con il Ball Tree, con foglie da 1 e da 32 punti, con i kernel SIMD e con quelli in C, e per le varianti OpenMP con 1 e con 4 thread. Per le varianti a 64 bit le etichette devono coincidere con quelle di scikit-learn punto per punto, e coincidono in tutti i casi; per le varianti a 32 bit, che lavorano in singola precisione, è ammessa una differenza fino allo 0,2% dei punti, per i motivi discussi nel Capitolo 7, ma nei casi di prova le etichette coincidono esattamente anche a 32 bit. Lo script verifica anche la gestione degli errori della riga di comando. Il programma è stato inoltre eseguito con Valgrind, senza errori di accesso alla memoria né memoria non liberata, e compilato con l'*Undefined Behavior Sanitizer* di GCC, senza segnalazioni.

6.3 Confronto tra le Varianti

La Tabella 6.2 e la Figura 6.1 confrontano le varianti del programma sul dataset di 20.000 punti in 16 dimensioni, aggiungendo un'ottimizzazione alla volta, insieme alla versione d'esame e a scikit-learn.

Variante	**Tempo (s)**	**Speedup**	**Distanze calcolate**
Versione d'esame	2,026	1,0×	–
Ricerca esaustiva, C	3,220	0,6×	399.980.000
Ricerca esaustiva, AVX	1,908	1,1×	399.980.000
Ball Tree, C	0,689	2,9×	66.992.612
Ball Tree, AVX	0,480	4,2×	66.992.612
Ball Tree, AVX, 4 thread	0,144	14,1×	66.992.612
Ball Tree, AVX, 8 thread	0,127	16,0×	66.992.612
scikit-learn, 1 thread	0,358	5,7×	–
scikit-learn, 8 thread	0,361	5,6×	–

Tabella 6.2: Varianti a confronto su 20.000 punti in 16 dimensioni; lo speedup è rispetto alla versione d'esame.

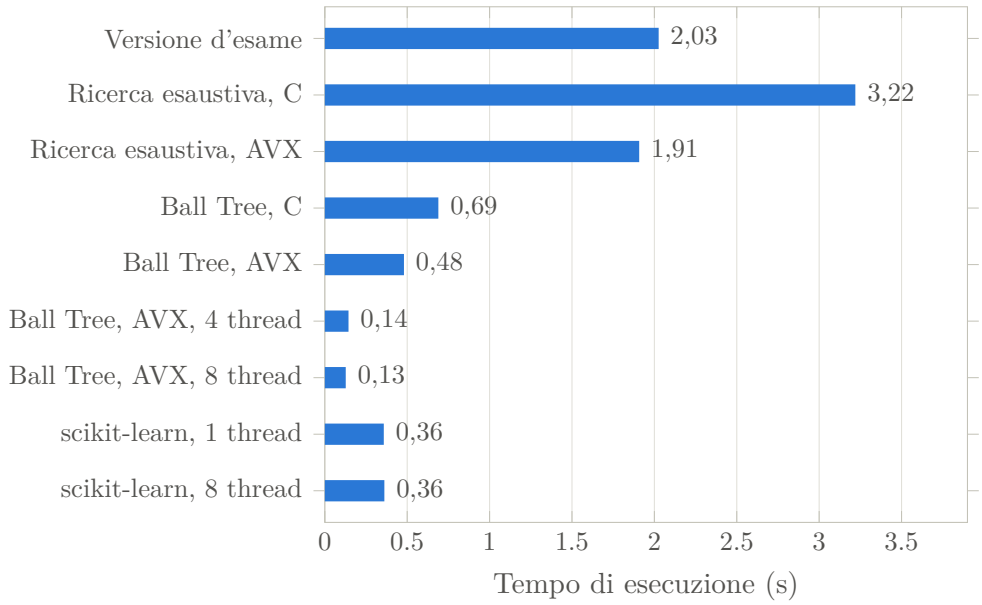


Figura 6.1: Tempo di esecuzione delle varianti su 20.000 punti in 16 dimensioni.

La lettura della tabella dall'alto verso il basso mostra il contributo di ciascuna tecnica:

- **Kernel SIMD.** A parità di algoritmo, i kernel AVX riducono il tempo di 1,7 volte con la ricerca esaustiva e di 1,4 volte con il Ball Tree, dove una parte del tempo va nella visita dell'albero e non nel calcolo delle distanze.
- **Ball Tree.** L'indice calcola sei volte meno distanze della ricerca esaustiva e riduce il tempo di quattro volte. Da solo, anche senza SIMD, porta il programma a quasi tre volte la velocità della versione d'esame.
- **OpenMP.** Con quattro thread, uno per core fisico, il tempo si riduce di 3,3 volte; con otto thread di 3,8 volte. Il programma completo è 16 volte più veloce della versione d'esame.
- **Confronto con scikit-learn.** Su un solo thread il programma è più lento di scikit-learn del 34%. scikit-learn con tutti i thread (`n_jobs=-1`) migliora di poco, perché la sua espansione dei cluster è sequenziale e le interrogazioni sono parallelizzate solo in parte. Con otto thread il programma è 2,8 volte più veloce.

La versione d'esame, compilata con `-O0` come prescritto dal corso, impiega poco più della ricerca esaustiva con i kernel AVX, benché usi un Ball Tree: le

foglie da 300 punti e le numerose allocazioni ne annullano quasi del tutto il vantaggio.

6.4 Confronto tra 32 e 64 Bit

La Tabella 6.3 confronta le due architetture sullo stesso dataset: a sinistra la variante a 32 bit, con kernel SSE in singola precisione, a destra quella a 64 bit, con kernel AVX in doppia precisione. L'ultima colonna è il rapporto tra i tempi: un valore minore di uno indica che la variante a 64 bit è più veloce.

Variante	**32 bit, SSE (s)**	**64 bit, AVX (s)**	**Rapporto**
Ricerca esaustiva, C	4,409	3,220	0,73
Ricerca esaustiva, SIMD	2,291	1,908	0,83
Ball Tree, C	0,918	0,689	0,75
Ball Tree, SIMD	0,576	0,480	0,83
Ball Tree, SIMD, 4 thread	0,168	0,144	0,86
Ball Tree, SIMD, 8 thread	0,149	0,127	0,85

Tabella 6.3: Variante a 32 bit e variante a 64 bit su 20.000 punti in 16 dimensioni.

La variante a 64 bit richiede dal 14 al 27% di tempo in meno in tutti i casi, benché lavori in doppia precisione e quindi con il doppio dei dati. Le due varianti elaborano lo stesso numero di valori per istruzione, quattro `float` in un registro XMM o quattro `double` in un registro YMM, e la singola precisione non porta quindi alcun vantaggio nel calcolo. Pesano invece due caratteristiche della modalità a 32 bit:

- la convenzione *cdecl* passa gli argomenti sullo stack e restituisce il risultato sullo stack x87, che il codice compilato con `-mfpmath=sse` deve poi riportare in un registro XMM passando per la memoria: ogni distanza attraversa la memoria due volte, nel kernel (`movss` e `fld`) e nel chiamante (`fstp` e `movss`);
- l'architettura a 32 bit dispone di 8 registri generali e 8 registri XMM, contro 16 e 16, e il compilatore deve salvare in memoria più variabili, come mostra il rallentamento anche delle varianti in C.

Una variante a 32 bit che usasse AVX in singola precisione elaborerebbe otto valori per istruzione, ma il corso prescriveva le sole istruzioni SSE per questa architettura.

6.5 Il Caso della Versione d'Esame

La relazione d'esame osservava che il Ball Tree non portava miglioramenti significativi rispetto alla ricerca esaustiva, e ne attribuiva la causa al costo di costruzione, alla dimensione delle foglie e allo sbilanciamento dell'albero. La causa principale era in realtà il dataset. In un ipercubo unitario a 50 dimensioni la distanza tra due punti casuali si concentra attorno a $\sqrt{50/6} \approx 2{,}9$: nel dataset dell'esame la media è 2,88, la deviazione standard 0,24 e la distanza massima 4,16. Con $\varepsilon = 5$ ogni punto ha quindi per vicini tutti gli altri, e il risultato è un unico cluster. Nessuna sfera dell'albero può essere disgiunta dall'intorno, e un indice che sa solo escludere le regioni lontane non può escludere nulla.

Variante	**Tempo (s)**	**Speedup**	**Distanze calcolate**
Versione d'esame	1,100	1,0×	–
Ricerca esaustiva, C	0,877	1,3×	24.995.000
Ricerca esaustiva, AVX	0,377	2,9×	24.995.000
Ball Tree, C	0,125	8,8×	5.053
Ball Tree, AVX	0,121	9,1×	5.053
Ball Tree, AVX, 4 thread	0,048	23,0×	5.053
Ball Tree, AVX, 8 thread	0,049	22,5×	5.053
scikit-learn, 1 thread	0,324	3,4×	–
scikit-learn, 8 thread	0,324	3,4×	–

Tabella 6.4: Varianti a confronto sul dataset dell'esame: 5000 punti uniformi in 50 dimensioni, $\varepsilon = 5$.

La Tabella 6.4 mostra l'effetto dell'accettazione in blocco delle sfere interamente contenute nell'intorno: per quasi ogni interrogazione la sfera della radice cade tutta dentro l'intorno, e il Ball Tree calcola 5053 distanze invece di 25 milioni. Il tempo che resta è speso quasi tutto nel copiare gli intorni, 25 milioni di indici

e 100 MB di memoria, ed è per questo che i kernel SIMD non vi incidono. Con la ricerca esaustiva, invece, il vantaggio dei kernel AVX sale a 2,3 volte, perché con 52 valori per riga ogni chiamata del kernel esegue 13 iterazioni e il costo fisso della chiamata pesa meno.

Il caso è istruttivo anche per un altro motivo: un dataset in cui tutti i punti formano un solo cluster non può rivelare errori nell'assegnazione delle etichette. Per questo i test e le misure di questo capitolo usano dataset con cluster reali e rumore, e il dataset dell'esame compare solo come caso limite.

6.6 Scalabilità con i Thread

La Figura 6.2 riporta lo speedup della prima fase parallela rispetto all'esecuzione su un thread, per il Ball Tree su 80.000 punti e per la ricerca esaustiva su 20.000 punti, entrambi in 16 dimensioni.

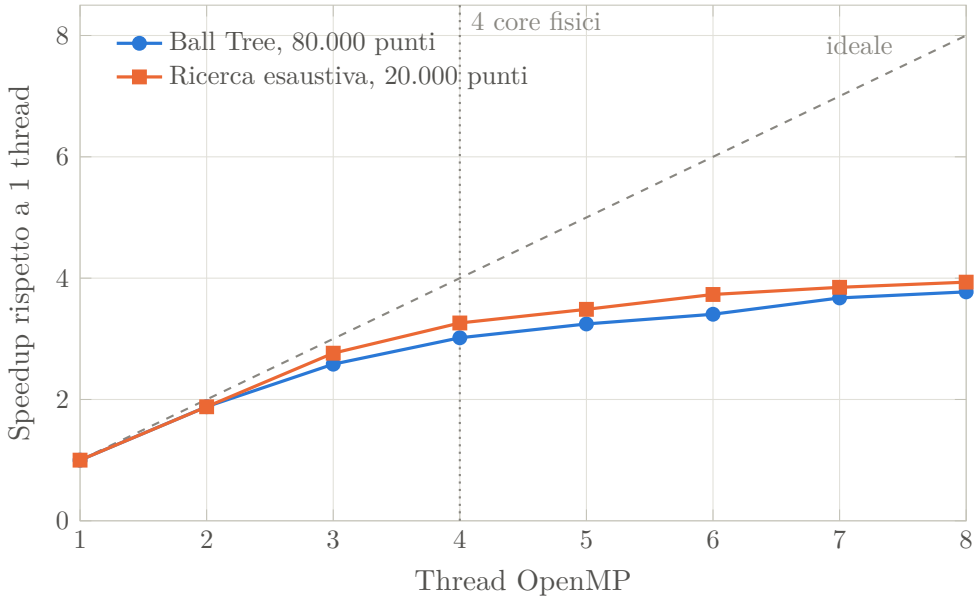


Figura 6.2: Speedup al crescere dei thread OpenMP. La linea tratteggiata è lo speedup ideale, la linea punteggiata il numero di core fisici.

Fino a quattro thread, uno per core fisico, lo speedup è di 3,0 per il Ball Tree e di 3,3 per la ricerca esaustiva. Non raggiunge il valore ideale per due motivi. Il primo è la frequenza del processore, che con un solo core attivo sale fino a 4 GHz e scende quando lavorano tutti i core. Il secondo, per il Ball Tree, è l'accesso alla memoria meno regolare della visita dell'albero, con i core in

competizione per la cache condivisa. Oltre quattro thread l'*hyper-threading* esegue due thread sullo stesso core fisico e aggiunge un altro 20–25%, fino a 3,8 e 3,9 con otto thread: i due thread di un core si alternano sulle stesse unità di calcolo e ne coprono i tempi di attesa.

La parte sequenziale è trascurabile: l'espansione dei cluster richiede circa 20 ms su 80.000 punti, lo 0,2% del tempo su un thread e l'1% su otto, e la costruzione del Ball Tree passa da 104 a 72 ms. Secondo la legge di Amdahl, una parte sequenziale pari all'1% del tempo pone allo speedup un limite teorico di 100.

6.7 Scalabilità con il Numero di Punti

La Figura 6.3 riporta il tempo di esecuzione al crescere del numero di punti da 5000 a 80.000, in 16 dimensioni e in scala logaritmica su entrambi gli assi, dove una retta di pendenza 2 indica un costo quadratico. La ricerca esaustiva si ferma a 40.000 punti, perché con 80.000 richiederebbe circa 40 secondi per esecuzione.

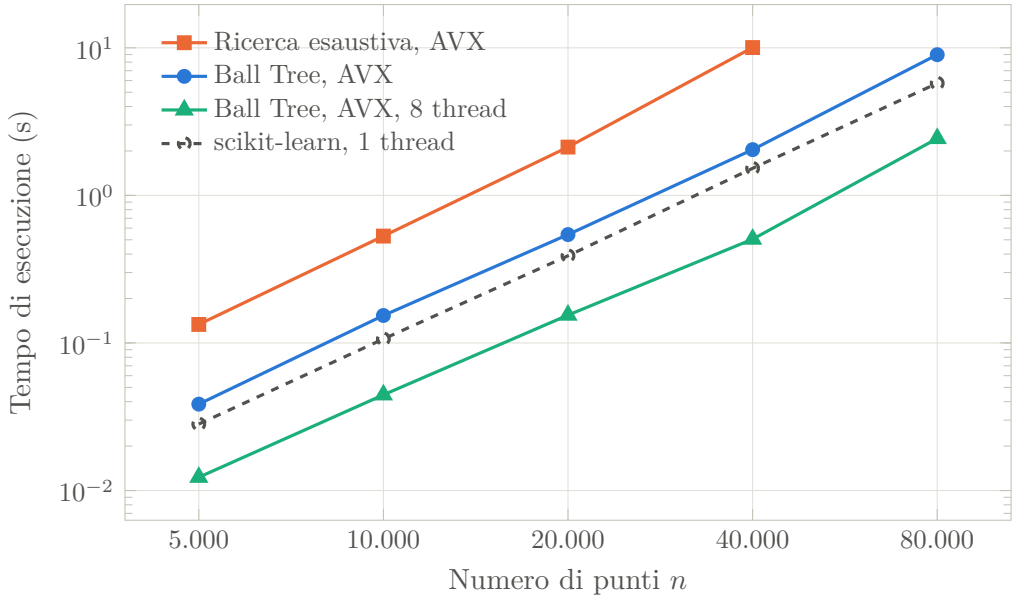


Figura 6.3: Tempo di esecuzione al crescere del numero di punti, in 16 dimensioni.

Tutte le curve hanno pendenza vicina a 2, compresa quella di scikit-learn: raddoppiando i punti il tempo quadruplica anche con il Ball Tree, e il numero di distanze calcolate passa da 4,3 milioni a un miliardo. Non è un difetto dell'indice

ma una proprietà dei dati: con un numero fisso di cluster, raddoppiare i punti raddoppia la densità e quindi la dimensione media degli intorni, mentre ε scelto con l'euristica diminuisce di poco. La dimensione complessiva del risultato cresce quindi in modo quadratico, e nessun algoritmo che elenchi gli intorni può fare meglio, come osservano Schubert et al. [2]. Il Ball Tree abbassa la costante, di un fattore compreso tra 3,5 e 5 rispetto alla ricerca esaustiva, e OpenMP la abbassa di un ulteriore fattore compreso tra 3 e 4.

6.8 Dimensione delle Foglie

Il parametro *leaf_size* regola un compromesso: foglie piccole producono un albero profondo, con più nodi da visitare e più distanze dai centri da calcolare; foglie grandi producono un albero basso ma costringono a esaminare molti punti lontani in ogni foglia raggiunta. La Figura 6.4 riporta il tempo in funzione della dimensione delle foglie, diviso per il tempo migliore di ciascun dataset.

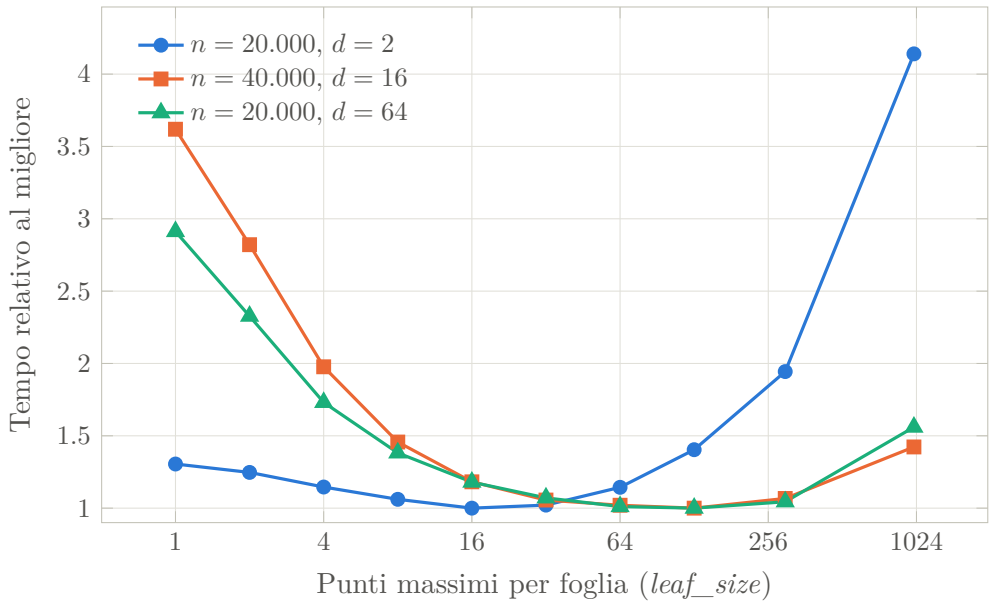


Figura 6.4: Tempo al variare della dimensione massima delle foglie, relativo al valore migliore di ciascun dataset.

Il valore migliore dipende dalla dimensionalità: 16 punti in due dimensioni, 128 in 16 e in 64 dimensioni. In poche dimensioni le sfere sono piccole e ben separate, e conviene scendere in profondità per escluderne di più; in molte dimensioni la potatura è comunque limitata e conviene ridurre il costo della

visita. Il valore predefinito, 32, resta entro l'8% dal migliore in tutti e tre i casi, mentre sia 128 sia valori molto piccoli penalizzano uno dei casi di oltre il 40%. Le 300 foglie della versione d'esame erano un valore ragionevole in molte dimensioni, ma avrebbero raddoppiato il tempo nel caso bidimensionale.

6.9 Effetto della Dimensionalità

La Tabella 6.5 e la Figura 6.5 confrontano il Ball Tree con la ricerca esaustiva su 20.000 punti al variare della dimensione, da 2 a 128.

d	**Esaustiva (s)**	**Ball Tree (s)**	**Speedup**	**Distanze**	**Profondità**
2	1,053	0,050	21,0×	1,3%	17
4	1,047	0,150	7,0×	6,2%	19
8	1,313	0,304	4,3×	13,7%	24
16	1,855	0,469	4,0×	16,7%	23
32	3,060	0,770	4,0×	18,3%	22
64	8,708	1,562	5,6×	19,2%	25
128	19,103	3,649	5,2×	20,5%	24

Tabella 6.5: Ball Tree e ricerca esaustiva al variare della dimensione, su 20.000 punti; la colonna Distanze riporta le distanze calcolate dal Ball Tree in percentuale di quelle della ricerca esaustiva.

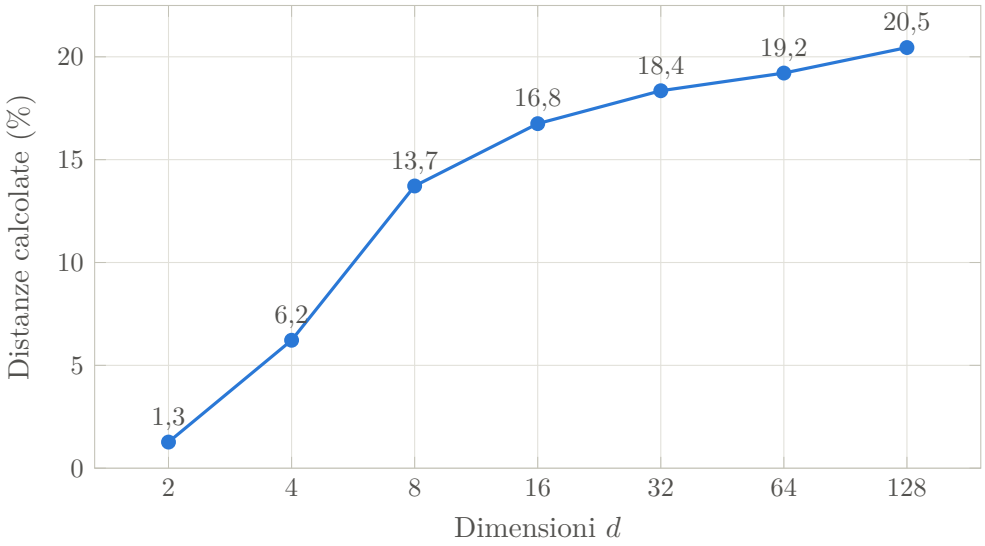


Figura 6.5: Distanze calcolate dal Ball Tree, in percentuale di quelle della ricerca esaustiva, al variare della dimensione.

In due dimensioni il Ball Tree calcola l'1,3% delle distanze ed è 21 volte più veloce della ricerca esaustiva. La percentuale cresce con la dimensione, com'è tipico degli indici spaziali: in molte dimensioni le distanze tendono a somigliarsi e le sfere si sovrappongono. La crescita però si arresta attorno al 20% invece di proseguire fino al 100%. Il motivo sta nella struttura dei dati: gli otto cluster restano ben separati in qualunque dimensione, e l'albero continua a escludere in blocco i cluster diversi da quello del punto interrogato, anche quando all'interno di un cluster non riesce più a potare. Con otto cluster di dimensioni simili resta da esaminare circa un ottavo dei punti, più il rumore e i nodi a cavallo di due cluster. Su dati senza alcuna struttura, come il dataset uniforme dell'esame con un ε più piccolo, la potatura in alta dimensione sarebbe molto minore.

6.10 SIMD Manuale e Ottimizzazioni del Compilatore

La Tabella 6.6 confronta i kernel in C e quelli SIMD con i due livelli di ottimizzazione: `-O0`, prescritto dal template del corso, e `-O2`, usato dalla versione attuale.

Indice	**Livello**	**C (s)**	**SIMD (s)**	**Vantaggio**
Ricerca esaustiva	`-O0`	16,618	2,949	5,63×
Ricerca esaustiva	`-O2`	3,131	1,889	1,66×
Ball Tree	`-O0`	3,269	0,767	4,26×
Ball Tree	`-O2`	0,691	0,471	1,47×

Tabella 6.6: Kernel in C e kernel SIMD con i livelli di ottimizzazione `-O0` e `-O2`, su 20.000 punti in 16 dimensioni.

Con `-O0` i kernel SIMD sembrano da 4,3 a 5,6 volte più veloci, ma il confronto è sbilanciato: senza ottimizzazioni il compilatore rilegge e riscrive in memoria ogni variabile a ogni iterazione, e la versione in C è molto più lenta del necessario. Con `-O2` il vantaggio scende a 1,5–1,7 volte, ed è il valore che misura davvero il contributo del codice scritto a mano: nasce dalle quattro somme parziali indipendenti che il compilatore non può introdurre da sé (Sezione 4.5). Anche il resto del programma beneficia delle ottimizzazioni: la visita del Ball Tree e la gestione delle liste, che non usano i kernel, fanno scendere del 40% il tempo del Ball Tree con i kernel SIMD tra `-O0` e `-O2`.

Capitolo 7

Limiti e Sviluppi Possibili

7.1 Memoria Occupata dagli Intorni

Il limite principale della formulazione in due fasi è la memoria: gli intorni dei punti core restano in memoria fino all'espansione, per un totale di quattro byte per ogni coppia di punti vicini. Sui dataset con cluster ben definiti usati nelle prove il costo è contenuto, ma quando ε è grande rispetto alla scala dei dati gli intorni diventano enormi: sul dataset uniforme dell'esame ogni punto ha per vicini tutti gli altri, e 5000 punti richiedono già 100 MB. Con 50.000 punti ne servirebbero 10 GB. scikit-learn condivide lo stesso limite, perché calcola tutti gli intorni in anticipo.
Due varianti ridurrebbero la memoria senza rinunciare al parallelismo:

- memorizzare solo gli archi tra punti core, più per ogni punto non core un solo punto core vicino, e calcolare le componenti connesse con una struttura *union-find*; è l'approccio di diverse implementazioni parallele di DBSCAN e renderebbe parallela anche l'espansione;
- elaborare i punti a blocchi, calcolando e consumando gli intorni di un blocco alla volta, al prezzo di interrogare di nuovo i punti al confine tra blocchi.

7.2 La Matrice delle Distanze

La relazione d'esame proponeva di precalcolare la matrice di tutte le distanze, per non calcolare mai due volte la stessa. Nella formulazione attuale il problema non si pone, perché ogni coppia viene esaminata al più due volte, una per ciascun punto, e le interrogazioni del Ball Tree ne escludono la maggior parte; la matrice richiederebbe inoltre n^2 valori, 20 GB in doppia precisione per 50.000 punti. Sfruttare la simmetria, calcolando ogni coppia una sola volta, dimezzerebbe il lavoro della ricerca esaustiva, ma complicherebbe la distribuzione del lavoro tra i thread.

7.3 Kernel SIMD

Il kernel della distanza ha un'unica catena di somme vettoriali: ogni iterazione deve attendere il risultato di `vaddpd` dell'iterazione precedente, con una latenza di quattro cicli sui processori della famiglia Skylake. Mantenere due o quattro accumulatori indipendenti, sommati alla fine, lascerebbe lavorare in parallelo le unità di calcolo e ridurrebbe il costo per dimensioni elevate; richiederebbe però di arrotondare la dimensione a un multiplo di otto o sedici valori. Altre estensioni offrirebbero ulteriori margini, a costo di restringere i processori supportati:

- le istruzioni FMA (*fused multiply-add*) fonderebbero moltiplicazione e somma in un'unica istruzione;
- AVX-512 raddoppierebbe il numero di valori per registro; il processore usato per le misure non la supporta.

Il costo di una chiamata al kernel, attraverso un puntatore a funzione, pesa infine sulle dimensioni piccole: con $d = 2$ una distanza richiede una sola iterazione del ciclo. Un kernel che confronti un punto con un'intera foglia in una sola chiamata ammortizzerebbe questo costo.

7.4 Struttura dell'Indice

Il Ball Tree riduce di molto le distanze calcolate in poche dimensioni, ma la sua efficacia cala al crescere della dimensione (Sezione 6.9), come per ogni indice spaziale. scikit-learn sceglie automaticamente tra un *k-d tree*, un Ball Tree e la ricerca esaustiva in base ai dati; una scelta analoga, insieme a una suddivisione dei nodi più accurata, per esempio lungo la direzione di massima varianza, migliorerebbe le prestazioni nei casi sfavorevoli. La relazione d'esame citava anche l'R-Tree, suggerito nel lavoro originale su DBSCAN, che è progettato per dati su disco e aggiornamenti dinamici, due esigenze assenti in questo contesto.

7.5 Precisione della Variante a 32 Bit

La variante a 32 bit lavora in singola precisione, come richiesto dal corso. Un punto la cui distanza da un vicino è pari a ε entro l'errore di arrotondamento può cadere dalla parte opposta rispetto al calcolo in doppia precisione, e cambiare il proprio stato di punto core o di confine. Per questo i test accettano per la variante a 32 bit una piccola percentuale di etichette diverse da quelle di scikit-learn, mentre per quella a 64 bit richiedono l'uguaglianza esatta; nei dataset di prova, generati con valori casuali, il caso non si è mai presentato, ma su dati con coordinate ripetute o ε pari a una distanza esistente non si può escludere.

Capitolo 8

Conclusioni

Il progetto ha realizzato DBSCAN in C e assembly x86 e lo ha accelerato combinando tre tecniche, ciascuna rivolta a un aspetto diverso del costo delle interrogazioni di vicinato. Sul dataset di riferimento, 20.000 punti in 16 dimensioni, il programma completo è sedici volte più veloce della versione consegnata all'esame e 2,8 volte più veloce di scikit-learn, producendo le stesse etichette punto per punto.

Il contributo di ciascuna tecnica

- **La struttura dati** è la leva più efficace: il Ball Tree riduce da tre a cinque volte il tempo rispetto alla ricerca esaustiva, e sui dati con ε grande, grazie all'accettazione in blocco delle sfere, di diversi ordini di grandezza. Anche i dettagli di implementazione contano: la sola copia dei punti in ordine di foglia riduce il tempo del 20–40%, un effetto paragonabile a quello dei kernel AVX, che con il Ball Tree lo riducono del 30%.

- **Le istruzioni SIMD** riducono il costo di ogni distanza di 1,4–2,3 volte, di più quando le righe sono lunghe. Anche con `-O2`, dove il compilatore vettorizza da sé, il codice scritto a mano conserva un vantaggio, perché può riordinare le somme in virgola mobile, cosa che al compilatore non è permessa.

- **OpenMP** porta uno speedup di 3 sui quattro core fisici e di quasi 4 con l'*hyper-threading*, ma solo dopo aver riformulato l'algoritmo: il

parallelismo va cercato al livello delle interrogazioni indipendenti e non all'interno della singola interrogazione, dove il costo della sincronizzazione supera il lavoro da distribuire.

Cosa ha insegnato la revisione

La revisione della versione d'esame ha mostrato quanto una misura possa ingannare. OpenMP non era mai stato attivato e il tempo era misurato come tempo di CPU, quindi ogni considerazione sulla parallelizzazione era priva di riscontro. Il dataset di prova conteneva un solo cluster, quindi non poteva rivelare errori nelle etichette né mostrare i benefici dell'indice. Due errori nei kernel assembly, la violazione della convenzione di chiamata a 64 bit e la variabile globale condivisa a 32 bit, erano invisibili nelle condizioni in cui il programma veniva provato. Il confronto sistematico con un'implementazione di riferimento su casi tipici e limite, eseguito automaticamente a ogni modifica, è ciò che rende affidabili i risultati di questa versione.

Sviluppi futuri

Gli sviluppi più promettenti, discussi nel Capitolo 7, sono tre:

- ridurre la memoria occupata dagli intorni, e rendere parallela anche l'espansione, con una struttura *union-find* sui punti core;
- usare più accumulatori nei kernel;
- scegliere l'indice spaziale in base alla dimensionalità dei dati.

Appendice A

Dalla Versione d'Esame alla Versione Attuale

La versione consegnata all'esame è conservata nel repository con l'etichetta `exam-2024` e si può compilare con `make exam`, che la costruisce come facevano gli script del corso. Questa appendice riassume le modifiche al template fornito dal docente e le differenze tra la versione d'esame e quella attuale.

A.1 Modifiche al Template

Il template del corso era stato pensato per un altro algoritmo e prevedeva un vettore di etichette in ingresso e un parametro `k`, il numero di elementi da estrarre. Già nella versione d'esame erano stati aggiunti i parametri *MinPts* ed ε alla struttura dei parametri e alla riga di comando, con i relativi controlli, e la funzione di lettura era stata estesa per completare le righe del dataset a un multiplo di quattro valori (Sezione 3.2). Il file delle etichette e il parametro `k` erano però rimasti obbligatori, benché ignorati: il valore di `k` veniva sovrascritto con il numero di punti. La versione attuale conserva il formato dei file del template ed elimina i parametri inutilizzati.

A.2 Errori Corretti

Problema nella versione d'esame	**Correzione**
OpenMP mai attivato: mancava `-fopenmp` negli script di compilazione	quattro varianti prodotte dal `Makefile`, con e senza `-fopenmp`
Kernel a 64 bit: registri ripristinati in ordine errato, `RBX` alterato	kernel che usano solo registri non da preservare
Kernel a 32 bit: risultato restituito tramite una variabile globale, condivisa tra i thread	cella sullo stack del thread
Tempo misurato con `clock()`, cioè come tempo di CPU sommato su tutti i thread	tempo reale con `clock_gettime`
File delle etichette con un elemento in più, letto oltre la fine del vettore	un'etichetta per punto
Controllo dei limiti in `getPuntiValue` con `>` invece di `>=`; estrazione da lista vuota non controllata	nuova `PointList`, usata solo in modi che non richiedono controlli
Ricorsione infinita nel Ball Tree con più di `LEAFSIZE` punti identici	nodo foglia quando il raggio è nullo
Memoria non liberata: albero, matrice originale dopo il padding, centroidi	ogni allocazione liberata; verificato con Valgrind
Mix di istruzioni AVX e SSE senza `vzeroupper`	solo istruzioni VEX e `vzeroupper` all'uscita
Intestazioni, nomi e messaggi ereditati da altri progetti (`cfs`, «Attention mechanism»)	nomi e commenti riscritti

Tabella A.1: Errori della versione d'esame e relative correzioni.

A.3 Modifiche Strutturali

- **Un solo albero di sorgenti** al posto di quattro copie del programma, diviso in moduli e compilato da un `Makefile`; commenti e identificatori in inglese.

- **Algoritmo in due fasi**, con la prima fase parallela e la regola di assegnazione dei punti di confine di scikit-learn.
- **Ball Tree** costruito per partizionamento sul posto, in parallelo, con i punti copiati in ordine di foglia e l'accettazione in blocco delle sfere interamente contenute nell'intorno.
- **Kernel SIMD** con interfaccia a puntatori, distanza al quadrato e uso anche per i centroidi.
- **Riga di comando** con opzioni per scegliere indice, kernel e numero di thread, e statistiche in formato JSON.
- **Verifica automatica**: script che confrontano le etichette con scikit-learn su casi tipici e limite, eseguiti anche dall'integrazione continua a ogni modifica, e script che riproducono tutte le misure di questa relazione.

Bibliografia

[1] M. Ester, H.-P. Kriegel, J. Sander, X. Xu, *A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise*, in Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD '96), AAAI Press, Portland, 1996, pp. 226–231.

[2] E. Schubert, J. Sander, M. Ester, H.-P. Kriegel, X. Xu, *DBSCAN Revisited, Revisited: Why and How You Should (Still) Use DBSCAN*, ACM Transactions on Database Systems, vol. 42, n. 3, 2017, art. 19. DOI: 10.1145/3068335.

[3] S. M. Omohundro, *Five Balltree Construction Algorithms*, Technical Report TR-89-063, International Computer Science Institute, Berkeley, 1989.

[4] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, É. Duchesnay, *Scikit-learn: Machine Learning in Python*, Journal of Machine Learning Research, vol. 12, 2011, pp. 2825–2830.

[5] Intel Corporation, *Intel 64 and IA-32 Architectures Software Developer's Manual*, volumi 1 e 2, e *Intel 64 and IA-32 Architectures Optimization Reference Manual*.

[6] H. J. Lu, M. Matz, M. Girkar, J. Hubička, A. Jaeger, M. Mitchell (a cura di), *System V Application Binary Interface, AMD64 Architecture Processor Supplement*, e *System V Application Binary Interface, Intel386 Architecture Processor Supplement*.

[7] OpenMP Architecture Review Board, *OpenMP Application Programming Interface*, versione 5.2, novembre 2021.